

QA Manager — Sistema de Gestión de Aseguramiento de Calidad

Informe final — Fase 4

Estudiante: Diego Andres Gomez Piamba
2026

1. Introducción

El presente informe documenta el desarrollo del prototipo web QA Manager, elaborado para la Fase 4. La solución se orienta a la gestión del aseguramiento de calidad de software mediante una interfaz navegable que permite administrar proyectos, requerimientos, casos de prueba, ejecuciones, defectos y reportes básicos.

2. Objetivo

Desarrollar un prototipo frontend funcional que permita demostrar un flujo de gestión de calidad de extremo a extremo: proyecto, requerimiento, caso de prueba, ejecución, resultado de prueba, registro de defecto y consulta de indicadores.

3. Alcance

El prototipo implementa interfaces completas y navegables, operaciones de creación, edición y eliminación, ejecución de casos, registro de bugs, KPI dinámicos, reportes básicos, persistencia local y diseño adaptable. El conjunto de funcionalidades se orienta a cubrir la mayor parte del alcance funcional definido para una demostración TRL 5.

4. Análisis de la solución

La necesidad abordada consiste en disponer de un espacio centralizado para organizar información de pruebas de software. La solución propuesta agrupa los elementos principales del ciclo QA y establece trazabilidad entre requerimientos, casos, ejecuciones y defectos.

5. Diseño

La aplicación utiliza una arquitectura frontend sencilla y mantenible. La interfaz se desarrolla con HTML5 y CSS3; la lógica de interacción se implementa en JavaScript y la persistencia se maneja mediante una capa de datos simulada que utiliza localStorage. La separación entre app.js y mock-api.js facilita la posterior sustitución de la fuente simulada por una API real o una base de datos.

6. Metodología de implementación

El desarrollo se organizó de forma incremental. Primero se definió la estructura del repositorio y la navegación; después se incorporaron las entidades del proceso QA; posteriormente se agregaron operaciones CRUD, ejecución de casos, generación de defectos y cálculo de indicadores; finalmente se documentaron el despliegue, los datos y las evidencias.

7. Funcionalidades implementadas

Se implementaron las siguientes funcionalidades principales: Dashboard, proyectos, requerimientos, casos de prueba, ejecución de casos, defectos, reportes, perfil y configuración. Los formularios incluyen validaciones básicas de campos obligatorios y los registros pueden mantenerse después de recargar el navegador.

8. Datos y persistencia

El prototipo utiliza datos simulados. No se incorporan datos bancarios reales ni credenciales reales. La aplicación inicializa un conjunto de datos de demostración y los guarda en localStorage. La carpeta data contiene la descripción del conjunto inicial y su procedencia simulada.

9. Validación del flujo principal

El flujo de demostración se puede realizar desde el menú: abrir un proyecto, revisar un requerimiento, crear o seleccionar un caso, ejecutarlo como Failed, registrar el defecto asociado y consultar los reportes. Al modificar registros, los KPI se recalculan con los datos actuales. También puede recargarse la página para comprobar la persistencia local.

10. Resultados

El resultado es un prototipo web funcional, navegable y adaptable, preparado para ser publicado mediante GitHub Pages. La solución dispone de operaciones de gestión sobre las principales entidades, datos simulados, indicadores calculados y documentación organizada en un repositorio con carpetas para código, datos, documentación, scripts, dashboard y evidencias.

11. Despliegue

El repositorio está preparado para GitHub Pages mediante un index.html en la raíz. También se incluye un workflow opcional en .github/workflows/pages.yml para automatizar el despliegue. La aplicación no requiere instalación de dependencias para su demostración básica.

12. Evidencias

La carpeta evidencias incluye una guía para capturar las pantallas reales del funcionamiento. Se recomienda registrar el login, dashboard, creación de proyecto, requerimientos, casos, ejecución fallida, bug, reportes y comportamiento responsive. Las capturas deben obtenerse directamente del prototipo ejecutándose y no deben presentarse imágenes ilustrativas como evidencia real.

13. Conclusiones

QA Manager integra en un único prototipo los elementos fundamentales del ciclo de aseguramiento de calidad. La implementación demuestra navegación, reglas básicas de interacción, gestión de datos simulados, trazabilidad y cálculo de indicadores. La arquitectura permite evolucionar el prototipo hacia una solución con backend, API y base de datos reales.

14. Trabajo futuro

Como evolución posterior se recomienda conectar una API REST real, incorporar autenticación segura, base de datos, control de permisos por roles, historial completo de cambios, filtros avanzados, exportación formal de reportes y almacenamiento de evidencias en servidor.

15. Matriz de cumplimiento del prototipo

Requisito	Implementación	Estado
Funciones principales implementadas	CRUD de proyectos, requerimientos, casos y bugs; ejecución; reportes	Cumple
Interfaces completas y navegables	Menú, dashboard y módulos funcionales	Cumple
Flujos extremo a extremo	Proyecto → Requerimiento → Caso → Ejecución → Bug → Reporte	Cumple
Validaciones frontend	Campos obligatorios y validaciones básicas	Cumple
Datos simulados	API simulada + localStorage	Cumple
Diseño responsive	CSS adaptable a escritorio, tablet y móvil	Cumple
Despliegue de pruebas	Preparado para GitHub Pages	Preparado